Day 96: Fine-Tuning the Mini-GPT on a Narrower Real Task

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 6 | Complete study guide

Watch: Fine-tuning Large Language Models Explained: AI Tutorial for Beginners (AIHub101) | GPT Models Explained: How Fine-Tuning Changes Everything! (Super Data Science)

Script: `day96_fine_tuning.py` · every number in this guide is copied by program from `day96_run_output.txt`, the real output of that script.

Day 92's mini-GPT never saw the last tenth of Tiny Shakespeare, which ends with a whole play: The Tempest. Starting from its trained weights, 400 steps on 24,258 characters of that play take the held-out loss on the play from 2.486 to 2.104 bits per character, while training the same model from scratch on the same text reaches only 4.807. The cost is forgetting: on other unseen Shakespeare the loss rises from 2.054 to 2.190. Mixing old text into every batch (replay) did best on both: 2.050 on the play and 2.089 elsewhere. All of this is one seed per run.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

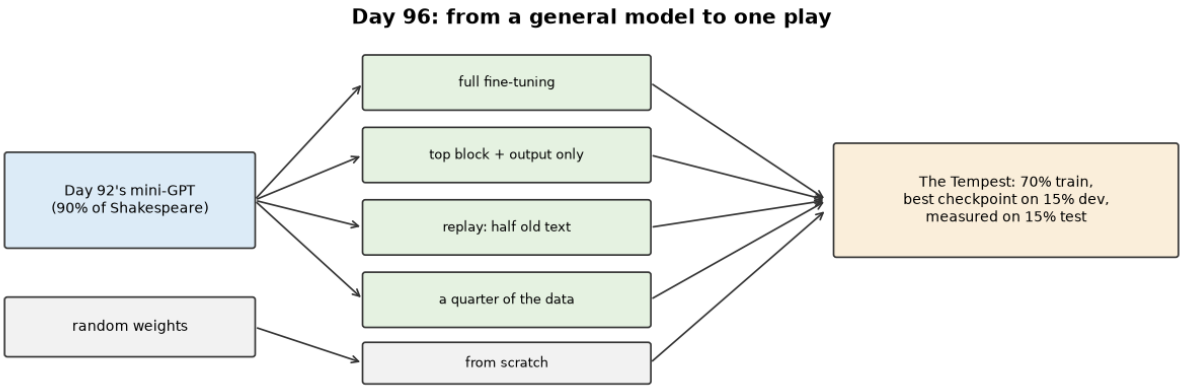
Fine-tuning means taking a model that has already been trained on a large, general text (**pretraining**) and training it a little more on a smaller, narrower text, usually with a smaller learning rate. The general knowledge is the starting point; the new text only has to teach what is different.

Catastrophic forgetting is the price: the weights that move to fit the new text were also doing the old job.

REAL-WORLD SCENARIO

An experienced nurse moving to a new hospital learns its forms and its doctors' names in a week; someone with no training could not learn nursing from those forms. And after years at the new place, some habits of the old one fade.

SYSTEM DIAGRAM



One general model, four ways to adapt it, and a from-scratch baseline; every run keeps its best checkpoint on the play's dev part and is measured on its test part.

Today's part for Day 97

`day96_exercise.py` holds the model-file and fine-tuning helpers Day 97 needs: `save_params` and `load_params` (nested weights to one file and back), `adam_subset` (Adam on named weights only) and `snapshot` (a deep copy for the best checkpoint). Its checks, real output:

ACTUAL OUTPUT

```
round trip: same names True, largest |difference| 0.0e+00
blocks.0.W   trainable: False  unchanged bit for bit: True
blocks.1.W   trainable: True   unchanged bit for bit: False
Wout         trainable: True   unchanged bit for bit: False
after training, the snapshot still equals the old values: True; the shallow copy does not (it
    shares the arrays): False
```

The last line is the reason `snapshot` exists: a shallow copy keeps pointing at arrays that Adam changes in place, so a 'best checkpoint' saved that way would silently become the latest weights.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The task: one play the model has never seen

DEFINITION — start here, no prior knowledge assumed

A fair fine-tuning test needs text the pretrained model never trained on. Day 90's split kept the last 10% of Tiny Shakespeare for evaluation; it ends with The Tempest. The play is split in its own order: 70% to train on, 15% dev (choosing the learning rate and the best checkpoint), 15% test (reported once). The rest of the held-out text, the end of The Taming of the Shrew, is never trained on by anyone and measures forgetting.

REAL-WORLD SCENARIO

Testing a translator on a book they have never read, with a few chapters to study first, and checking afterwards that they have not got worse at everything else.

SPECIFICATION

Model	Day 92's saved mini-GPT: 4 blocks, width 128, 4 heads, context 128, 824,129 weights.
Measure	Day 92's held-out bits per character with full context (lower is better).
New speakers	speaker names (capitals and a colon, alone on a line) in the play but never in the training text.

Why this test

Zero-shot numbers first: what the general model already knows about the new play before any fine-tuning.

IMPLEMENTATION

```
def task_data():
    tr_text, va_text = d90.load_text()
    s = va_text.index(START_MARK)
    play, rest = va_text[s:], va_text[:s]
    a, b = int(SPLITS[0] * len(play)), int((SPLITS[0] + SPLITS[1]) * len(play))
    return tr_text, play[:a], play[a:b], play[b:], rest
```

ACTUAL OUTPUT

```
The Tempest starts 1,080,739 characters into Tiny Shakespeare and runs 34,655 characters to the end of the
    file. Split in order: train 24,258, dev 5,198, test 5,199. The rest of the held-out text (the end of The
    Taming of the Shrew, 76,885 characters) measures forgetting.
every character of the play is in the model's vocabulary: True
speakers of the play who never speak in the training text: 11 of 11: ADRIAN, ALONSO, ANTONIO, ARIEL, CALIBAN,
    FERDINAND, FRANCISCO, GONZALO, MIRANDA, PROSPERO, SEBASTIAN
zero-shot, Day 92's model as it is (held-out bits per character, full context):
  The Tempest dev 2.819, test 2.486; rest of the held-out text 2.054
Actual output from this lesson's run.
```

What it means

Every one of the play's 11 of 11 speakers is new to the model: it has never seen PROSPERO or MIRANDA at the start of a line. Zero-shot, the play is clearly harder than other unseen Shakespeare: 2.486 bits on its test part against 2.054 on the rest. The play's dev part is harder still (2.819), so dev and test numbers are not comparable with each other, only across models.

WHY CHOOSE THIS?

A real distribution shift with no leakage: the play is not in the pretraining text at all.

WHY NOT THIS? (TRADEOFFS)

The play's three parts differ in difficulty, and 5,199 test characters is a small test set.

2. The fine-tuning learning rate, chosen on dev

DEFINITION — start here, no prior knowledge assumed

Full fine-tuning updates every weight. Three peak learning rates, each a fraction of Day 92's pretraining peak (0.003), each run for 400 steps with a 20-step warm-up and Day 92's cosine decay, checked on dev every 25 steps. The best checkpoint by dev loss is kept.

REAL-WORLD SCENARIO

Turning a recipe you know into a new dish: small changes taste better than starting the recipe over.

MATHEMATICS

Adam on the trainable set K only: $\theta_k \leftarrow \theta_k - \eta_t\, m_k / (\sqrt{v_k} + \varepsilon)$ *for k in K; every other weight is unchanged*

(m, v: Adam's bias-corrected averages)

η_t *= Day 92's schedule: linear warm-up to the peak, then a cosine down to 0.1 × peak*

IMPLEMENTATION

```
def adam_subset(P, G, state, lr, keys, clip=1.0):
    g = {k: v for k, v in d90.w12.leaves(G) if k in keys}
    scale = min(1.0, clip / np.sqrt(sum(np.sum(a ** 2) for a in g.values())))
    state["t"] += 1; t = state["t"]
    for k, A in d90.w12.leaves(P):
        if k not in keys:
            continue
        m = state["m"].setdefault(k, np.zeros_like(A)); v = state["v"].setdefault(k,
            np.zeros_like(A))
        gk = g[k] * scale
        m[:] = 0.9 * m + 0.1 * gk; v[:] = 0.999 * v + 0.001 * gk ** 2
        A -= lr * (m / (1 - 0.9 ** t)) / (np.sqrt(v / (1 - 0.999 ** t)) + 1e-8)
```

ACTUAL OUTPUT

```
 peak lr  best dev  at step  dev at the end
  0.0001     2.218      400           2.218
  0.0003     2.212      100           2.347
   0.001     2.294       50           3.071
chosen: peak learning rate 0.0003
Actual output from this lesson's run.
```

What it means

0.0003 won, at step 100; after that its dev loss rose to 2.347: with only 24,258 characters the model starts memorising within a few hundred steps, and keeping the best checkpoint is what saves the result. 0.001 was best at step 50 and ended at 3.071, worse than zero-shot. 0.0001 was still improving at step 400 (best at the last check), only 2.218 against 2.212: a longer run at the smallest rate might win, and a gap of 0.006 bits from one seed each is not a clear winner. Not tested.

WHY CHOOSE THIS?

Choosing on dev keeps the test part untouched for the final comparison.

WHY NOT THIS? (TRADEOFFS)

Three rates and one seed: the choice between the two smaller rates is close to noise.

3. Six ways to get a model of The Tempest

DEFINITION — start here, no prior knowledge assumed

The chosen full fine-tuning run is compared with: zero-shot; **from scratch** (the same model with random weights, trained only on the play, at the pretraining peak 0.003); **top block + output only** (everything else frozen); **replay** (half of every batch from the original training text); and full fine-tuning on **a quarter of the data**. All 400 steps, best dev checkpoint, four runs at once in separate processes.

REAL-WORLD SCENARIO

Six ways to prepare a new employee: hire someone experienced or a beginner, let them change all their habits or only a few, keep them doing some old work while learning, or give them less material to learn from.

SPECIFICATION

Same for all	400 steps, batches of 16 windows of 128 characters, seed 0, dev check every 25 steps.
Trained weights	all 824,129 weights, or 206,145 for the last block and the output layer.
Forgetting	bits per character on the rest of the held-out text, never trained on by any run.

Why this test

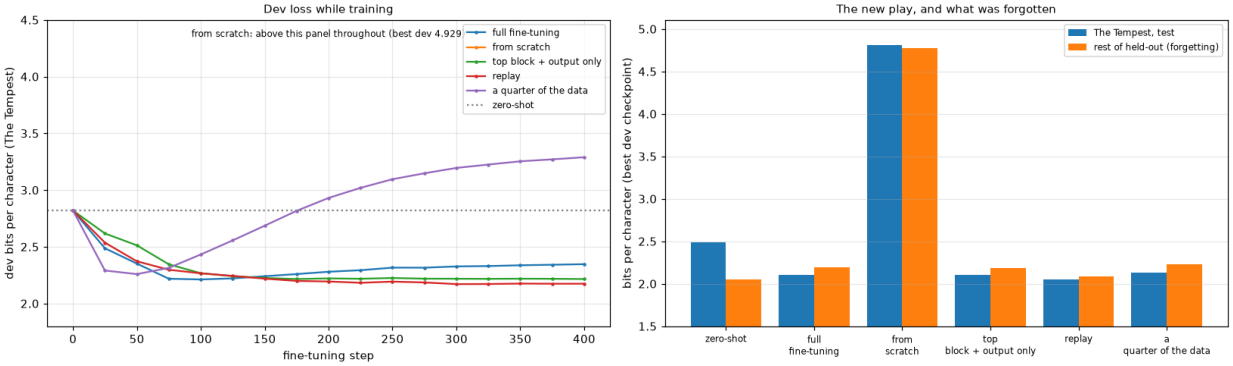
Each variant isolates one choice: whether pretraining helps at all (from scratch), how much of the model has to move (top only), whether forgetting can be reduced (replay), and how much new data is needed (a quarter).

IMPLEMENTATION

```
for step in range(1, STEPS + 1):
    o = rng.randint(0, len(tr) - T - 1, B)
    x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
    if replay:
        o2 = rng.randint(0, len(old) - T - 1, B // 2)
        x[B // 2:] = np.stack([old[i:i + T] for i in o2]); y[B // 2:] = np.stack([old[i + 1:i + T
            + 1] for i in o2])
    _, G = d90.gpt_loss_grads(x, y, P, h=h)
    adam_subset(P, G, state, d92.lr_schedule(step - 1, lr, warmup=WARMUP, total=STEPS), keys)
    if step % CHECK_EVERY == 0:
        d = d92.eval_bits(P, dev, T, h); curve.append((step, d))
        if d < best[0]:
            best = (d, step, snapshot(P))
```

ACTUAL OUTPUT

```
model                       trained weights best step     dev    test  rest of held-out
zero-shot (Day 92 as is)                 0         -   2.819   2.486             2.054
full fine-tuning                    824129       100   2.212   2.104             2.190
from scratch                        824129       100   4.929   4.807             4.773
top block + output only             206145       400   2.216   2.105             2.185
replay                              824129       300   2.171   2.050             2.089
a quarter of the data               824129        50   2.259   2.130             2.230
Actual output from this lesson's run.
```



Left: dev loss on The Tempest every 25 steps. Right: test loss and forgetting at each run's best dev checkpoint.

What it means

Pretraining is most of the result. From scratch, the best checkpoint is at step 100 and the test loss is 4.807 bits, far worse than even the untouched model (2.486). With 824129 weights and 24 thousand characters, 400 steps from random weights can neither learn the language nor avoid memorising; a smaller model or a longer schedule might do better (not tested).

Moving a quarter of the weights was enough. Top block + output only (206145 weights) reached 2.105 on test against 2.104 for full fine-tuning: no measurable difference, and it was still improving at step 400. Its forgetting was about the same (2.185 against 2.190).

Replay did best on both measures: 2.050 on the play and 2.089 on the rest, only a little above the zero-shot 2.054. That it also helped on the play itself was not expected; a likely reason is that the old text acts as a regulariser against memorising 24 thousand characters, but that was not tested, and with one seed per run the difference of about 0.05 bits should be confirmed before relying on it.

A quarter of the data (2.130) cost only about 0.03 bits on test, but its best checkpoint came at step 50 and its dev loss then climbed back above the zero-shot line (left panel): with so little text, the checkpoint rule is doing all the work. It also forgot the most.

WHY CHOOSE THIS?

Fine-tuning turns a small amount of new text into a large gain, cheaply.

WHY NOT THIS? (TRADEOFFS)

Every fine-tuned model forgot some general skill; only replay kept it close, at the cost of reading old data.

4. What fine-tuning changes in the text

DEFINITION — start here, no prior knowledge assumed

The loss is an average; the samples show what it buys. Both models continue the same 20 prompts from the play's test part with Day 93's top-p 0.9 sampler. Counted: how many of the speaker lines they write name one of the play's eleven new speakers.

REAL-WORLD SCENARIO

After a week at the new hospital, the nurse starts using the right doctors' names.

IMPLEMENTATION

```
for name, texts in out.items():
    sp = [m for t in texts for m in re.findall(r"^([A-Z][A-Z' ]+):$", t, re.M)]
    share = np.mean([s in new for s in sp]) if sp else 0.0
    print(f"  {name:<20}: {len(sp)} speaker lines, {share:.3f} of them name a speaker new in The
        Tempest")
```

ACTUAL OUTPUT

```
Both models continue the same 20 prompts from the play's test part by 300 characters (top-p 0.9,
    Day 93's sampler, seed 0). How many of the speaker lines they write name one of the play's own
    new speakers?
  Day 92 (zero-shot)  : 19 speaker lines, 0.000 of them name a speaker new in The Tempest
  full fine-tuning    : 36 speaker lines, 0.778 of them name a speaker new in The Tempest
prompt 1 ('excel the golden age.\n\nSEBASTIAN'), first 160 characters:
--- Day 92 (zero-shot) ---
US:
I think your goodness man and in are
Now to drink'd of thy deposed and like weary.
Ah, thou art thou bount to my blass
Of his case, had it is weaping thing
--- full fine-tuning ---
:
I feather, besiege you do
That he saw aboved fit hath made you to the
knees of his stranged, the wood of you.
PROSPERO:
Master,
And be cheek, I'll knowled th
Actual output from this lesson's run.
```

What it means

The untouched model wrote 19 speaker lines, none naming a Tempest speaker: it can only use names it saw. After fine-tuning, 0.778 of its 36 speaker lines name one, PROSPERO among them in the first sample. The text is still not good Shakespeare; what changed is the play's own vocabulary and layout. The counts come from one sampling seed.

WHY CHOOSE THIS?

A direct, checkable sign of adaptation that the loss alone does not show.

WHY NOT THIS? (TRADEOFFS)

A narrow measure: it says nothing about whether the lines make sense.

Interview preparation — technical questions

Q: How do you choose a fine-tuning learning rate?

A: Try a few values below the pretraining rate on a dev set; here 0.0003 won, with 0.0001 close behind and still improving.

Q: Why keep the best checkpoint?

A: With little data the dev loss turns up within a few hundred steps; 0.0003 went from 2.212 at step 100 to 2.347 at the end.

Q: How do you freeze part of a model?

A: Update only a named subset of weights; adam_subset skips every other key, and the exercise checks they do not move by one bit.

Q: What is replay?

A: Mixing original training data into fine-tuning batches; here half of each batch, which cut forgetting from 2.190 to 2.089.

Q: Why does a deep copy matter for checkpoints?

A: Adam updates arrays in place; a shallow copy shares them and silently changes.

Interview preparation — theoretical questions

Q: Why does pretraining help so much here?

A: The play is mostly ordinary English and Shakespeare's style; only names and topics are new. From scratch, 24 thousand characters cannot teach that.

Q: What is catastrophic forgetting?

A: Losing old skills as the weights move to fit new data: 2.054 to 2.190 on other unseen Shakespeare here.

Q: When is fine-tuning only the top layers enough?

A: When the new task needs new outputs more than new features; here it matched full fine-tuning.

Q: What would parameter-efficient methods like LoRA add?

A: Even fewer trained numbers, by learning small low-rank changes; not built here.

Key takeaway and what's next

KEY TAKEAWAY

Starting from a pretrained model is what makes a small dataset useful: 400 steps took the new play from 2.486 to 2.104 bits, where training from scratch reached 4.807. Most of the risk is overfitting and forgetting, handled here by a dev-chosen checkpoint and by replaying old text.

- Zero-shot 2.486; full fine-tuning 2.104; top block + output only 2.105; replay 2.050; a quarter of the data 2.130; from scratch 4.807.
- Forgetting (rest of held-out, zero-shot 2.054): 2.190, 2.185, 2.089, 2.230 for the same four fine-tuned runs.

- Learning rate: 0.0003 chosen (2.212 dev), 0.0001 close (2.218) and still improving.
- Speaker lines naming a new Tempest speaker: 0.000 before, 0.778 after.

Suggested watch order, if starting from zero

- 1. Fine-tuning Large Language Models Explained: AI Tutorial for Beginners (AIHub101) — what fine-tuning is and why it works.
- 2. GPT Models Explained: How Fine-Tuning Changes Everything! (Super Data Science) — how fine-tuning changes a GPT-style model.

Day 97 preview

- **Week 14's build: language-model evaluation.** The week's six parts assembled into one evaluation of Day 92's and today's models: perplexity, and BLEU and ROUGE written from scratch, tested against controls.